REPORTE DE PRÁCTICA NO. 6

Analizador Sintactico

ALUMNO: PALMA VELAZQUEZ ARIADNA BETSABE /CORTES SANTOS MARIA FERNANDA/ JACOME CAYATANO JOSE DAVID

Dr. Eduardo Cornejo-Velázquez

1. *Introducción*

En el presente documento se expone el desarrollo de un analizador sintáctico implementado mediante el uso de las herramientas Flex y Yacc. El objetivo principal de este trabajo es construir un sistema capaz de reconocer y procesar correctamente distintos elementos léxicos y sintácticos de expresiones, tales como números enteros, números decimales, identificadores, paréntesis, así como operadores aritméticos que incluyen suma, resta, multiplicación y división. Asimismo, se contempla el manejo de operadores comparativos, permitiendo ampliar la capacidad de análisis del sistema. A lo largo del documento se describe la estructura del código, la definición de tokens y reglas gramaticales, así como el funcionamiento general del analizador, evidenciando la integración entre el análisis léxico y sintáctico para la correcta interpretación de las expresiones de entrada.

2. *Marco Teórico*

Una ***Gramática Formal*** definida sobre un alfabeto $\sum$ es una tupla de la forma:

$$\boldsymbol{G} = \{\sum_{\boldsymbol{T}}, \sum_{\boldsymbol{N}}, \boldsymbol{S},\ \boldsymbol{P}\}$$

donde:

$\sum_{\boldsymbol{T}}$ es un alfabeto de símbolos terminales
$\sum_{N}$ es un alfabeto de símbolos no terminales
$\boldsymbol{S}$ es el símbolo inicial de la gramática
$\boldsymbol{P}$ es un conjunto de producciones gramaticales

Además, se cumple:

$S \in \sum_{N}$
$\sum_{T} \cap \sum_{N} = \emptyset$
$\sum = \sum_{T} \cup \sum_{N}$

La gramática formal $\boldsymbol{G}$ permite generan un lenguaje $L = \{x \in \sum_{T}^{*} | S \overset{*}{\rightarrow} x\}$
Por lo tanto, las palabras del lenguaje estarán formadas por cadenas de símbolos terminales generadas a partir del símbolo inicial de la gramática utilizando las producciones que la definen.

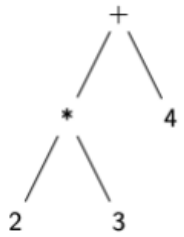
Una ***expresión regular*** es una notación normalizada para representar lenguajes regulares, es decir, lenguajes generados por gramáticas regulares (Tipo 3). Las expresiones regulares permiten describir con exactitud y sencillez cualquier lenguaje regular.

El analizador sintáctico es la parte del compilador que se encarga de analizar el código de entrada en base a una gramática dada y, en caso de que el programa de entrada sea válido, construye el árbol sintáctico que lo reconoce.

En teoría, se supone que la salida del analizador sintáctico es alguna representación del árbol sintáctico que reconoce la secuencia de tokens suministrada por el analizador léxico, pero en la práctica, el analizador sintáctico también identifica errores cuando se producen y se comunica con la tabla de símbolos para almacenar información de los procesos realizados.

Un analizador sintáctico es un programa que analiza códigos para determinar su estructura sintáctica.

Ejemplo de análisis sintáctico aritmético: La estructura sintáctica de la cadena **2 * 3 + 4** es el siguiente árbol:



El análisis sintáctico forma parte del preprocesamiento en la mayoría de las aplicaciones reales.
Consideramos expresiones aritméticas:

- construidas con números, operaciones **(+ y ∗)** y paréntesis.
- **+ y ∗** asocian por la derecha.
- ∗ tiene más prioridad que **+**.

3. *Herramientas empleadas*

-Equipo de cómputo con software para programación a elección de los alumnos.
-Programa Flex
-Archivo Lex
-Archivo Yacc/Bios

4. Desarrollo

A continuación se presenta la explicación de los codigos desarrollados tanto en la parte Lexica como de Yacc para poder crear una analizador sintactico. En la siguiente imagen 1.1 se puede observar que se presenta el desarrollo de la parte del analizador Lexico donde se declaran las reglas de composición de los tokens solicitados en la practica; los cuales son identificadores, enteros, decimales, comparadores, suma, resta, división, multiplicación y parentesis. Tambien se declara ahi la union de ambos archivos con la libreria y.tab.h para que el archivo de yacc pueda transferir de forma mas rapida los tokens validos del archivo lexico.

```
%{
    #include <stdio.h>
    #include <string.h>
    #include "y.tab.h"
%}

%%
[0-9]+                    { yylval.valor_entero = atoi(yytext); return ENTERO; }
[0-9]*\.[0-9]+            { yylval.valor_decimal = atof(yytext); return DECIMAL; }
[a-zA-Z_][a-zA-Z0-9_]* { yylval.variable = strdup(yytext); return IDENTI; }
"\+"                      { return SUMA; }
"-"                       { return RESTA; }
"\*"                      { return MULTI; }
"/"                       { return DIVI; }
"\("                      { return PARENA; }
"\)"                      { return PARENC; }
"=="|"!="|"<="|">="|"<"|">" { yylval.condicional = strdup(yytext);return COMPARA; }
\n {return EOL;}
[ \t]                     { /* Ignorar espacios y tabuladores */ }
%%

int yywrap(void){return 1;}
```

Posteriormente despues de terminar con la parte del analizador lexico se inicia el archivo yacc/bios declarando la libreria yylex(), la cual es la encargada de mandar a llamar los tokens del otro archivo. Se utiliza de la funcio "union" para indicarle al analizador sintactico que no solo va a recibir valores int durante la compilación del archivo; a cada tipo de valor se le da una variable que sera utilizada a la hora de colocar los nombres de los tokens para indicar que esas variables son las que van a traer un valor diferente al predifinido que es el int con el que viene el yacc/bios. Asi como tambien se indica con "left" el oredn gerarquico con el que tienen que trabajar el analizador; y con "type" se le indica que la expresion ingresada debe de permitir el ingreso de los decimales.

```
%{
        #include<stdio.h>
        int yylex(void);
        int yyerror(const char *s);
%}

%union{
int valor_entero;
float valor_decimal;
char* variable;
char* condicional;
}
%token <valor_entero> ENTERO
%token <valor_decimal> DECIMAL
%token <condicional> COMPARA
%token <variable> IDENTI
%token SUMA RESTA MULTI DIVI PARENA PARENC EOL

%left SUMA RESTA
%left MULTI DIVI

%type <valor_decimal> expression
```

En esta ultima parte del archivo de yacc/bios se puede ver que se declararon las reglas gramaticales que debe de serguir el analizador para indicar que es valida o no la expresion; aqui el orden en que se colocan las expresiones se decidieron de forma albitraria por el equipo no era necesario seguir un orden en especifico; tambien se puede notar que estamos manejando la recursividad hacia la izquierda y de forma desendente. Cabe mencionar que en nuestro main se tenia que colocar la opción de que el archivo leyera un archivo txt pero como equipo no logramos resolver el problema de la lectura de un archivo txt en el analizador; ya que si lo leia pero no imprimia los resultados en la pantalla de la consola.

```
%%

statements : statement statements
           | statement
           ;
statement  : expression EOL {printf("-> EXPRESION ARITMETICA VALIDA\n\n");}
           | expression COMPARA expression EOL {printf("op_condicinal -> %s\n -> EXPRESION CONDICIONAL VALIDA \n\n", $2);}
           | error EOL      { yyerrok;}
           | EOL            { }
           ;


expression : ENTERO {$$=$1; printf("entero -> %d\n",(int)$$);}
           | DECIMAL {$$=$1; printf("decimal -> %f\n",$$);}
           | IDENTI { $$=0.0; printf("identificador -> %s\n",$1);}
           | expression SUMA expression {$$=$1+$3; printf("suma -> +\n Resultado:%f\n", $$);}
           | expression RESTA expression {$$=$1-$3; printf("resta -> -\n Resultado:%f\n", $$);}
           | expression MULTI expression {$$=$1*$3; printf("multiplicacion-> * \n Resultado:%f\n", $$);}
           | expression DIVI expression {if ($3 == 0.0) {
                   printf("-> Error matematico: Division por cero\n");
                   $$ = 0.0;
               } else {
                   $$ = $1/$3; printf("division -> / \n Resultado:%f\n", $$);
               }
             }
           | PARENA expression PARENC   { $$ = $2; printf("paren_abre -> (\n paren_cierre -> )\n");}
           ;
%%

int main(){
yyparse();
return 0;
}
int yyerror(const char *s) {
    printf("-> EXPRESION INVALIDA\n\n");
    return 0;
}
```

Pruebas Validas

```
C:\Users\Ferny-KFC\Desktop\

2*(3+4)
entero -> 2
entero -> 3
entero -> 4
suma -> +
 Resultado:7.000000
paren_abre -> (
 paren_cierre -> )
multiplicacion-> *
 Resultado:14.000000
-> EXPRESION ARITMETICA VALIDA

(x+10)*2
identificador -> x
entero -> 10
suma -> +
 Resultado:10.000000
paren_abre -> (
 paren_cierre -> )
entero -> 2
multiplicacion-> *
 Resultado:20.000000
-> EXPRESION ARITMETICA VALIDA

(x+10)*2.5
identificador -> x
entero -> 10
suma -> +
 Resultado:10.000000
paren_abre -> (
 paren_cierre -> )
decimal -> 2.500000
multiplicacion-> *
 Resultado:25.000000
-> EXPRESION ARITMETICA VALIDA

8<8
entero -> 8
entero -> 8
op_condicinal -> <
 -> EXPRESION CONDICIONAL VALIDA
```

C:\Users\Ferny-KFC\Desktop\

```
multiplicacion-> *
 Resultado:0.000000
entero -> 2
division -> /
 Resultado:0.000000
-> EXPRESION ARITMETICA VALIDA

area_base * altura +2
identificador -> area_base
identificador -> altura
multiplicacion-> *
 Resultado:0.000000
entero -> 2
suma -> +
 Resultado:2.000000
-> EXPRESION ARITMETICA VALIDA

w
identificador -> w
-> EXPRESION ARITMETICA VALIDA

ljgbaiurg
identificador -> ljgbaiurg
-> EXPRESION ARITMETICA VALIDA

4
entero -> 4
-> EXPRESION ARITMETICA VALIDA
```

C:\Users\Ferny-KFC\Desktop\

```
10.51-9.5
decimal -> 10.510000
decimal -> 9.500000
resta -> -
 Resultado:1.010000
-> EXPRESION VALIDA

9-1.5
entero -> 9
decimal -> 1.500000
resta -> -
 Resultado:7.500000
-> EXPRESION VALIDA

5*5
entero -> 5
entero -> 5
multiplicacion-> *
 Resultado:25.000000
-> EXPRESION VALIDA

1.5*2.5
decimal -> 1.500000
decimal -> 2.500000
multiplicacion-> *
 Resultado:3.750000
-> EXPRESION VALIDA

12*0.1
entero -> 12
decimal -> 0.100000
multiplicacion-> *
 Resultado:1.200000
-> EXPRESION VALIDA

10/10
entero -> 10
entero -> 10
division -> /
 Resultado:1.000000
-> EXPRESION VALIDA
```

```
C:\Users\Ferny-KFC\Desktop\

x + 5
identificador -> x
entero -> 5
suma -> +
 Resultado:5.000000
-> EXPRESION ARITMETICA VALIDA

area_base * altura /2
identificador -> area_base
identificador -> altura
multiplicacion-> *
 Resultado:0.000000
entero -> 2
division -> /
 Resultado:0.000000
-> EXPRESION ARITMETICA VALIDA

area_base * altura +2
identificador -> area_base
identificador -> altura
multiplicacion-> *
 Resultado:0.000000
entero -> 2
suma -> +
 Resultado:2.000000
-> EXPRESION ARITMETICA VALIDA
```

```
5 + * 3
entero -> 5
-> EXPRESION INVALIDA

8 +
entero -> 8
-> EXPRESION INVALIDA

5 + 3)
entero -> 5
entero -> 3
suma -> +
 Resultado:8.000000
-> EXPRESION INVALIDA

()
-> EXPRESION INVALIDA

a < b < c
identificador -> a
identificador -> b
-> EXPRESION INVALIDA

if (a < b) 5 + 5
identificador -> if
-> EXPRESION INVALIDA
```